

Debounce vs Throttling in JavaScript

Feature	Debounce	Throttling
Definition	Ensures a function is executed only after a specified delay has passed since the last call.	Ensures a function is executed at most once in a specified interval.
Execution Timing	Executes after the user stops triggering for a given time interval.	Executes immediately and then at most once every interval.
Use Case Example	Search box input (API call after typing stops), Auto-save handling.	Scroll handling, mouse move (drag & drop), preventing button click spam.
Control	Prevents multiple rapid executions, runs only once after inactivity.	Controls execution rate, allows periodic execution without over-restriction.
Analogy	Wait until user finishes talking before responding.	Respond at most once every X seconds while user is talking.
When to Use	Expensive tasks that should run only once after user stops input.	Tasks that should run at controlled frequency while action continues.

Code Examples

Debounce Implementation:

```
function debounce(fn, delay){
  let timer;
  return function(...args){
    clearTimeout(timer);
    timer = setTimeout(()=>fn.apply(this,args), delay);
  };
}
```

Throttle Implementation:

```
function throttle(fn, limit){
  let lastCall = 0;
  return function(...args){
    const now = Date.now();
    if(now - lastCall >= limit){
      lastCall = now;
      fn.apply(this,args);
    }
  };
}
```

Common Interview Questions and Model Answers

Q: What is Debouncing?

A: Debouncing is a technique where a function is executed only after a specified delay has passed since the last time it was called. Useful for search input fields, where execution should only occur after typing stops.

Q: What is Throttling?

A: Throttling ensures a function is executed at most once in a given time interval, even if it is called multiple times. Commonly applied to scroll or resize events.

Q: Difference between Debouncing and Throttling?

A: Debouncing → Executes after inactivity. Throttling → Executes at controlled intervals while action continues.

Q: Can you give real-life examples of both?

A: Debouncing → Search bar autocomplete, form validation. Throttling → Infinite scrolling, window resize, button click spam prevention.

Q: Which one would you use for an input field where the user types continuously?

A: Debouncing, since the goal is to trigger the function only after the user stops typing.

Q: Which one would you use for a window resize handler?

A: Throttling, because UI updates should occur periodically while resizing continues.

Q: Can Debouncing and Throttling be combined?

A: Yes, in scenarios where immediate execution is needed at first (throttle) and a final execution after the user stops (debounce). This hybrid approach balances responsiveness and performance.

Q: What's the difference between timestamp-based and timeout-based throttling?

A: Timestamp-based executes immediately if enough time has passed, while timeout-based schedules execution at the end of the interval.